API Reference & DTOs

Every endp

Student	Samson Odongo
ID	SSAMODO111111
Course	INSY 492 — Senior Project
Base URL (dev)	http://localhost:5000
Base URL (prod)	https://api.aiminutes.app
Auth	JWT via httpOnly cookie | Header: not required (cookie is sent automatically)
Date	April 2026

GET POST PUT PATCH DELETE ■ = Auth required ■ = Public ✓ = Required field

Table of Contents

1.	**AUTH** — /api/auth/*	Register, login, logout, refresh token
2.	**MEETINGS** — /api/meetings/*	Create, list, get, upload audio
3.	**TRANSCRIPTS** — /api/meetings/{id}/transcript	Get raw transcript
4.	**MINUTES** — /api/meetings/{id}/minutes/*	Get, edit, approve, send, export minutes
5.	**ARCHIVE** — /api/minutes/search	Full-text search across all minutes
6.	**ORGANISATION** — /api/org/*	Org details, members, recipients, branding
7.	**ADMIN** — /api/admin/*	Platform-wide admin — orgs, users, plans
8.	**WEBHOOKS** — /webhooks/*	SendGrid delivery webhooks
9.	**DTOs — REQUEST** — All write DTOs	Every field used in POST/PUT bodies
10.	**DTOs — RESPONSE** — All read DTOs	Every field returned in API responses
11.	**ENUM VALUES** — Shared enums	MeetingStatus, PlanTier, UserRole
12.	**ERROR SHAPES** — All error codes	Standard error body returned on 4xx/5xx

1. AUTH — Registration, login, token management

POST `/api/auth/register` — Free

Register a new user and create their organization

Request body — RegisterDto

Field	Type	Req?	Description
`OrgName`	*string*	✓	Name of the organization (2–100 chars)
`Email`	*string*	✓	User's email address — must be unique
`Password`	*string*	✓	Min 8 chars, 1 uppercase, 1 number, 1 special char
`ConfirmPassword`	*string*	✓	Must match Password exactly
`FullName`	*string*	✓	User's display name (2–100 chars)

Response body — AuthResponseDto

Field	Type	Req?	Description
`UserId`	*Guid*	—	Newly created user ID
`OrgId`	*Guid*	—	Newly created organization ID
`Email`	*string*	—	Echoes back the registered email
`FullName`	*string*	—	User's display name
`Role`	*string*	—	Always 'OrgOwner' for new registrations
`Plan`	*string*	—	Always 'Free' for new registrations

JWT is set as httpOnly cookie named jwt — not returned in body.

HTTP responses

Status	Meaning
201	User and org created successfully — JWT cookie set
400	Validation failed — passwords don't match, email invalid, etc.
409	Email already registered

POST `/api/auth/login` — Free

Authenticate and receive a JWT cookie

Request body — LoginDto

Field	Type	Req?	Description
`Email`	*string*	✓	Registered email address
`Password`	*string*	✓	Account password

Response body — AuthResponseDto

Field	Type	Req?	Description
`UserId`	*Guid*	—	Authenticated user ID
`OrgId`	*Guid*	—	User's organization ID
`Email`	*string*	—	User's email address
`FullName`	*string*	—	User's display name
`Role`	*string*	—	Admin | OrgOwner | Member
`Plan`	*string*	—	Free | Pro | Enterprise

HTTP responses

Status	Meaning
200	Login successful — JWT cookie set
400	Missing or invalid request body
401	Wrong email or password

POST `/api/auth/logout` — Free

Clear the JWT cookie and invalidate session

No request body required.

HTTP responses

Status	Meaning
200	Cookie cleared successfully — body: { message: 'Logged out' }
401	Not authenticated

POST `/api/auth/refresh` — Free

Re-issue JWT — call when token is close to expiry

No request body. Reads current JWT from cookie, validates it, and issues a fresh 8-hour token.

Response body — AuthResponseDto (same as login)

HTTP responses

Status	Meaning
`200`	New JWT set in cookie
`401`	Token expired or invalid — user must log in again

GET `/api/auth/me` — Free

Get the currently authenticated user's profile

No request body.

Response body — UserProfileDto

Field	Type	Req?	Description
`UserId`	*`Guid`*	—	User ID
`OrgId`	*`Guid`*	—	Organization ID
`Email`	*`string`*	—	User's email
`FullName`	*`string`*	—	User's display name
`Role`	*`string`*	—	Admin | OrgOwner | Member
`Plan`	*`string`*	—	Free | Pro | Enterprise
`MeetingsUsedThisMonth`	*`int`*	—	Count toward free tier limit (Free plan only)
`MeetingsLimit`	*`int`*	—	5 for Free, -1 (unlimited) for Pro/Enterprise

HTTP responses

Status	Meaning
`200`	User profile returned
`401`	Not authenticated

2. MEETINGS — Create, list, get meeting sessions, upload audio

POST `/api/meetings` — Free

Create a new meeting session

Request body — CreateMeetingDto

Field	Type	Req?	Description
`Title`	*string*	✓	Meeting title (3–255 chars)
`ScheduledAt`	*DateTime?*	—	Optional — when meeting occurred. Defaults to now if omitted
`Attendees`	*string[]?*	—	Optional list of attendee names to pre-populate
`Description`	*string?*	—	Optional context about the meeting (max 500 chars)

Response body — MeetingDto

Field	Type	Req?	Description
`Id`	*Guid*	—	Meeting ID — used in all subsequent requests
`OrgId`	*Guid*	—	Organization this meeting belongs to
`Title`	*string*	—	Meeting title
`Status`	*string*	—	Always 'created' on first creation
`ScheduledAt`	*DateTime?*	—	Meeting date/time (UTC)
`CreatedAt`	*DateTime*	—	Record creation timestamp (UTC)
`CreatedByName`	*string*	—	Display name of the user who created this meeting

HTTP responses

Status	Meaning
201	Meeting created — Location header: /api/meetings/{id}
400	Validation error — title missing or too long
401	Not authenticated
402	Free plan: monthly meeting limit reached

GET `/api/meetings` — Free

List all meetings for the authenticated user's organization

Query parameters

Field	Type	Req?	Description
`page`	*int?*	—	Page number, 1-based. Default: 1
`pageSize`	*int?*	—	Items per page. Default: 20, max: 100
`status`	*string?*	—	Filter by status (e.g. pending_validation, approved)
`from`	*string?*	—	ISO date filter: meetings on or after this date (UTC)
`to`	*string?*	—	ISO date filter: meetings on or before this date (UTC)

Response body — PagedResult<MeetingListItemDto>

Field	Type	Req?	Description
`items`	*MeetingListItemDto[]*	—	Array of meeting summaries (see below)
`totalCount`	*int*	—	Total number of meetings matching the filter
`page`	*int*	—	Current page number
`pageSize`	*int*	—	Items per page
`totalPages`	*int*	—	Total number of pages

MeetingListItemDto fields

Field	Type	Req?	Description
`Id`	*Guid*	—	Meeting ID
`Title`	*string*	—	Meeting title
`Status`	*string*	—	Current pipeline status (see Enum Values, section 11)
`ScheduledAt`	*DateTime?*	—	When the meeting occurred
`CreatedAt`	*DateTime*	—	Record creation time
`HasMinutes`	*bool*	—	True if AI minutes have been generated
`ApprovedAt`	*DateTime?*	—	When minutes were approved (null if not yet approved)

HTTP responses

Status	Meaning
200	Paged list returned
401	Not authenticated

GET `/api/meetings/{id}` — Free

Get a single meeting's full details and current status

Path parameter id: Guid — the meeting ID

Response body — MeetingDetailDto

Field	Type	Req?	Description
`Id`	*Guid*	—	Meeting ID
`OrgId`	*Guid*	—	Organization ID
`Title`	*string*	—	Meeting title
`Description`	*string?*	—	Optional meeting description
`Status`	*string*	—	Current status — poll this to track pipeline progress
`ScheduledAt`	*DateTime?*	—	Meeting date/time
`CreatedAt`	*DateTime*	—	Record creation time
`EndedAt`	*DateTime?*	—	When recording was stopped (null if not yet ended)
`AudioPath`	*string?*	—	Internal audio file path — for display only, not downloadable
`AudioDurationSecs`	*int?*	—	Duration of the recorded audio in seconds
`CreatedByName`	*string*	—	Name of the user who created the meeting
`ApprovedByName`	*string?*	—	Name of user who approved minutes (null if not approved)
`ApprovedAt`	*DateTime?*	—	When minutes were approved
`HasTranscript`	*bool*	—	True once transcription is complete
`HasMinutes`	*bool*	—	True once AI has generated draft minutes

HTTP responses

Status	Meaning
200	Meeting detail returned
401	Not authenticated
403	Meeting belongs to a different organization
404	Meeting not found

POST `/api/meetings/{id}/audio` — Free

Upload recorded audio file — triggers background processing pipeline

Content-Type: multipart/form-data

Form fields

Field	Type	Req?	Description
`file`	*IFormFile*	✓	Audio file. Accepted: .mp3 .m4a .wav .webm .ogg. Max size: 100MB
`durationSecs`	*int?*	—	Optional duration in seconds — stored for display

Response body — AudioUploadResponseDto

Field	Type	Req?	Description
`MeetingId`	*Guid*	—	The meeting ID audio was uploaded to
`Status`	*string*	—	Always 'uploading' immediately after upload
`Message`	*string*	—	Human-readable confirmation message
`EstimatedWaitSeconds`	*int*	—	Rough estimate of processing time

Returns HTTP 202 Accepted immediately — do not wait for transcription. Poll GET /api/meetings/{id} to track progress.

HTTP responses

Status	Meaning
202	Audio received — background job queued
400	File missing, wrong type, or exceeds 100MB
401	Not authenticated
404	Meeting not found
409	Audio already uploaded to this meeting

DELETE `/api/meetings/{id}` — Free

Soft-delete a meeting and all associated data

No request body. Only the OrgOwner or Admin can delete meetings.

HTTP responses

Status	Meaning
204	Meeting deleted — no body returned
401	Not authenticated
403	Only OrgOwner or Admin can delete meetings
404	Meeting not found

3. TRANSCRIPTS — Access raw transcription data

GET `/api/meetings/{id}/transcript` — Free

Get the raw Whisper transcript with speaker segments

Response body — TranscriptDto

Field	Type	Req?	Description
`Id`	*Guid*	—	Transcript record ID
`MeetingId`	*Guid*	—	Meeting this transcript belongs to
`RawText`	*string*	—	Full plain-text transcript (all speakers combined)
`CreatedAt`	*DateTime*	—	When transcription completed
`Segments`	*TranscriptSegmentDto[]*	—	Speaker-labelled segments (see below)

TranscriptSegmentDto fields

Field	Type	Req?	Description
`Speaker`	*string*	—	Speaker label e.g. 'Speaker 1' or name if identified
`Text`	*string*	—	What this speaker said in this segment
`StartMs`	*int*	—	Start time of segment in milliseconds from audio start
`EndMs`	*int*	—	End time of segment in milliseconds

HTTP responses

Status	Meaning
200	Transcript returned
401	Not authenticated
404	Meeting not found or transcript not yet generated

4. MINUTES — Draft, validate, approve, send and export meeting minutes

GET `/api/meetings/{id}/minutes` — **Free**

Get the AI-generated draft or approved minutes

Response body — MinutesDto

Field	Type	Req?	Description
`Id`	*Guid*	—	Minutes record ID
`MeetingId`	*Guid*	—	Meeting these minutes belong to
`Status`	*string*	—	pending_validation | approved | sent
`Version`	*int*	—	Increments on each edit. Starts at 1
`DraftJson`	*object*	—	Raw AI output — structured JSON (see MinutesDraftDto below)
`ApprovedHtml`	*string?*	—	Final HTML content after user approval. Null until approved
`ApprovedBy`	*Guid?*	—	ID of user who approved. Null until approved
`ApprovedByName`	*string?*	—	Name of approver. Null until approved
`ApprovedAt`	*DateTime?*	—	Approval timestamp (UTC). Null until approved
`CreatedAt`	*DateTime*	—	When AI generated the minutes
`UpdatedAt`	*DateTime*	—	Last edited timestamp

MinutesDraftDto — structure of the DraftJson field

Field	Type	Req?	Description
`MeetingTitle`	*string*	—	Meeting title extracted by AI
`MeetingDate`	*string*	—	Date as extracted from transcript context
`Attendees`	*string[]*	—	List of identified attendee names
`AgendaItems`	*string[]*	—	List of agenda topics discussed
`DiscussionPoints`	*DiscussionItem[]*	—	Key points from each discussion (see below)
`Decisions`	*string[]*	—	Explicit decisions made during the meeting
`ActionItems`	*ActionItemDto[]*	—	Action items with owner and optional due date
`NextMeeting`	*string?*	—	Next meeting info if mentioned in discussion

ActionItemDto fields

Field	Type	Req?	Description
`Task`	*string*	—	Description of the action item
`Owner`	*string*	—	Person responsible for this action
`DueDate`	*string?*	—	Due date as mentioned in meeting (free text, not parsed)
`Done`	*bool*	—	Checkbox state — toggled during validation. Default: false

DiscussionItem fields

Field	Type	Req?	Description

`Topic`	*string*	—	Discussion topic heading
`Summary`	*string*	—	Summary of what was discussed under this topic

HTTP responses

Status	Meaning
200	Minutes returned
401	Not authenticated
404	Meeting has no minutes yet — AI generation not complete

PUT **`/api/meetings/{id}/minutes`** **Free**

Save edits made in the validation editor

Request body — UpdateMinutesDto

Field	Type	Req?	Description
`DraftJson`	*object*	✓	Full updated MinutesDraftDto object — replace the whole draft
`EditSummary`	*string?*	—	Optional note about what was changed (stored in version history)

Response body — MinutesDto (same as GET, reflects saved state)

HTTP responses

Status	Meaning
200	Minutes saved — version incremented
400	DraftJson missing or malformed
401	Not authenticated
403	Cannot edit already-approved minutes
404	Minutes not found

POST `/api/meetings/{id}/minutes/approve` — **Free**

Approve the draft minutes — locks them from further editing

Request body — ApproveMinutesDto

Field	Type	Req?	Description
`FinalHtml`	*string*	✓	The final HTML content rendered from the TipTap editor. This is what will be stored and emailed. Must be sanitised before sending.

Response body — MinutesDto with status: approved

HTTP responses

Status	Meaning
200	Minutes approved — status set to approved
400	FinalHtml is empty or missing
401	Not authenticated
409	Minutes already approved

POST `/api/meetings/{id}/minutes/send` **Pro**

Email the approved minutes to the org's recipient list

Request body — SendMinutesDto

Field	Type	Req?	Description
`Subject`	*string?*	—	Email subject line. Default: 'Minutes: {MeetingTitle}'
`AdditionalNote`	*string?*	—	Optional text appended before the minutes body in the email
`OverrideRecipients`	*string[]?*	—	If provided, send ONLY to these emails (ignores org list). Useful for re-sending to a subset.

Response body — SendMinutesResponseDto

Field	Type	Req?	Description
`SentAt`	*DateTime*	—	Timestamp when email was dispatched
`RecipientCount`	*int*	—	Number of recipients the email was sent to
`SendGridMessageId`	*string*	—	SendGrid message ID for delivery tracking

HTTP responses

Status	Meaning
200	Email dispatched — delivery is async, check webhooks for confirmation
400	Minutes not approved yet — must approve before sending
401	Not authenticated
402	Pro plan required
404	Meeting or minutes not found
409	Minutes already sent — use override to re-send

GET `/api/meetings/{id}/minutes/export` **Pro**

Download approved minutes as PDF or DOCX

Query parameters

Field	Type	Req?	Description
`format`	*string*	✓	'pdf' or 'docx' — determines the file format returned

Response: File download — Content-Type: application/pdf or application/vnd.openxmlformats-officedocument.wordprocessingml.document

HTTP responses

Status	Meaning
200	File returned as binary download
400	format parameter missing or invalid
401	Not authenticated
402	Pro plan required
404	Minutes not found or not yet approved

5. ARCHIVE — Full-text search across all org minutes

GET `/api/minutes/search` **Pro**

Full-text search across all approved minutes in the organization

Query parameters

Field	Type	Req?	Description
`q`	*string?*	—	Search keyword(s). Uses PostgreSQL full-text search (tsvector). Supports AND: 'budget report', phrase: '"action items"'
`from`	*string?*	—	ISO date: only return minutes from meetings on or after this date
`to`	*string?*	—	ISO date: only return minutes from meetings on or before this date
`attendee`	*string?*	—	Filter: only minutes where this name appears in attendees list
`page`	*int?*	—	Page number. Default: 1
`pageSize`	*int?*	—	Items per page. Default: 20, max: 50

Response body — PagedResult<MinutesSearchResultDto>

Field	Type	Req?	Description
`items`	*MinutesSearchResultDto[]*	—	Array of search results
`totalCount`	*int*	—	Total matching records
`page`	*int*	—	Current page
`totalPages`	*int*	—	Total pages

MinutesSearchResultDto fields

Field	Type	Req?	Description
`MeetingId`	*Guid*	—	ID of the meeting these minutes belong to
`MeetingTitle`	*string*	—	Meeting title
`MeetingDate`	*DateTime?*	—	When the meeting occurred
`ApprovedAt`	*DateTime*	—	When these minutes were approved
`Attendees`	*string[]*	—	Attendee names extracted by AI
`Snippet`	*string*	—	Highlighted text snippet showing where the keyword matched
`ActionItemCount`	*int*	—	Number of action items in these minutes

HTTP responses

Status	Meaning
200	Search results returned (empty array if no matches)
401	Not authenticated

`402`	Pro plan required for archive search

6. ORGANISATION — Org profile, members, email recipients, settings

GET `/api/org` — Free

Get org profile, plan details, and usage stats

Response body — OrgDto

Field	Type	Req?	Description
`Id`	*Guid*	—	Organization ID
`Name`	*string*	—	Organization name
`Plan`	*string*	—	Free | Pro | Enterprise
`PlanRenewsAt`	*DateTime*	—	When the plan usage counters reset
`MeetingsUsedThisMonth`	*int*	—	Number of meetings started this calendar month
`MeetingsLimit`	*int*	—	5 for Free, -1 (unlimited) for Pro/Enterprise
`MemberCount`	*int*	—	Total number of active members in the org
`CreatedAt`	*DateTime*	—	When the org was created

HTTP responses

Status	Meaning
200	Org details returned
401	Not authenticated

PUT `/api/org` — Free

Update organization name and settings

Request body — UpdateOrgDto (OrgOwner or Admin only)

Field	Type	Req?	Description
`Name`	*string?*	—	New organization name (2–100 chars). Omit to leave unchanged

Response body — OrgDto (same as GET)

HTTP responses

Status	Meaning
200	Org updated
400	Validation failed
401	Not authenticated
403	Only OrgOwner or Admin can update org

GET `/api/org/members` — Pro

List all members in the organization

Response body — OrgMemberDto[]

Field	Type	Req?	Description
`UserId`	*Guid*	—	Member's user ID
`FullName`	*string*	—	Member's display name
`Email`	*string*	—	Member's email address
`Role`	*string*	—	OrgOwner | Member
`JoinedAt`	*DateTime*	—	When they joined the org
`IsActive`	*bool*	—	False if the member has been deactivated

HTTP responses

Status	Meaning
200	Member list returned
401	Not authenticated
402	Pro plan required

POST `/api/org/members/invite` — **Pro**

Invite a new member by email — sends an invitation email

Request body — InviteMemberDto (OrgOwner only)

Field	Type	Req?	Description
`Email`	*string*	✓	Email address to invite
`FullName`	*string*	✓	Display name for the invited member
`Role`	*string?*	—	Member | OrgOwner. Default: Member

Response body — InvitationDto

Field	Type	Req?	Description
`InvitationId`	*Guid*	—	Invitation record ID
`Email`	*string*	—	Invited email
`ExpiresAt`	*DateTime*	—	Invitation expiry (48 hours from now)
`Status`	*string*	—	Always 'pending' immediately after invite

HTTP responses

Status	Meaning
201	Invitation created and email sent
400	Email missing or invalid
401	Not authenticated
402	Pro plan required
403	Only OrgOwner can invite members
409	This email is already a member or has a pending invitation

DELETE **/api/org/members/{userId}** Pro

Remove a member from the organization

No request body. OrgOwner or Admin only. Cannot remove the last OrgOwner.

HTTP responses

Status	Meaning
204	Member removed
400	Cannot remove the last OrgOwner
401	Not authenticated
402	Pro plan required
403	Insufficient permissions
404	Member not found

GET **/api/org/recipients** Pro

List all configured email distribution recipients

Response body — EmailRecipientDto[]

Field	Type	Req?	Description
Id	*Guid*	—	Recipient record ID
Email	*string*	—	Recipient email address
Name	*string*	—	Recipient display name

HTTP responses

Status	Meaning
200	Recipient list returned
401	Not authenticated
402	Pro plan required

PUT `/api/org/recipients` **Pro**

Replace the entire email distribution list — full overwrite

Request body — UpdateRecipientsDto (OrgOwner only)

Field	Type	Req?	Description
`Recipients`	*RecipientInputDto[]*	✓	Full new list. This replaces the existing list entirely. Pass empty array to clear all.

RecipientInputDto fields

Field	Type	Req?	Description
`Email`	*string*	✓	Valid email address
`Name`	*string*	✓	Display name — shown in email 'To:' field

Response body — EmailRecipientDto[] (the new full list)

HTTP responses

Status	Meaning
200	Recipients updated
400	One or more email addresses are invalid
401	Not authenticated
402	Pro plan required
403	Only OrgOwner can update recipients

7. ADMIN — Platform administration — Admin role required on all endpoints

GET `/api/admin/orgs` — Admin

List all organizations on the platform

Query parameters

Field	Type	Req?	Description
`page`	*int?*	—	Page number. Default: 1
`pageSize`	*int?*	—	Default: 50, max: 200
`plan`	*string?*	—	Filter by plan: Free | Pro | Enterprise

Response body — PagedResult<AdminOrgDto>

Field	Type	Req?	Description
`Id`	*Guid*	—	Org ID
`Name`	*string*	—	Org name
`Plan`	*string*	—	Current plan
`MemberCount`	*int*	—	Number of members
`MeetingsAllTime`	*int*	—	Total meetings ever created
`MeetingsThisMonth`	*int*	—	Meetings in current calendar month
`CreatedAt`	*DateTime*	—	Org creation date

HTTP responses

Status	Meaning
200	Org list returned
401	Not authenticated
403	Admin role required

GET `/api/admin/orgs/{orgId}` — Admin

Get detailed info about a specific org

Response is **AdminOrgDetailDto** — same as AdminOrgDto plus a member list and recent meeting list.

HTTP responses

Status	Meaning
200	Org detail returned
401	Not authenticated
403	Admin role required
404	Org not found

PUT `/api/admin/orgs/{orgId}/plan` — Admin

Change an organization's plan tier

Request body — ChangePlanDto

Field	Type	Req?	Description
`Plan`	*string*	✓	New plan: Free | Pro | Enterprise
`Reason`	*string?*	—	Optional internal reason for the change — logged in audit trail

Response body — OrgDto with updated plan

Changing plan automatically re-issues JWT on the user's next request (token refresh).

HTTP responses

Status	Meaning
200	Plan changed
400	Invalid plan value
401	Not authenticated
403	Admin role required
404	Org not found

GET `/api/admin/users` — Admin

List all users on the platform

Query parameters: page, pageSize, orgId (filter by org), role (filter by role)

Response body — PagedResult<AdminUserDto>

Field	Type	Req?	Description
`UserId`	*Guid*	—	User ID
`FullName`	*string*	—	Display name
`Email`	*string*	—	Email address
`OrgId`	*Guid*	—	Organization ID
`OrgName`	*string*	—	Organization name
`Role`	*string*	—	Admin | OrgOwner | Member
`CreatedAt`	*DateTime*	—	Account creation date
`IsActive`	*bool*	—	False if account has been deactivated

HTTP responses

Status	Meaning
200	User list returned
401	Not authenticated
403	Admin role required

8. WEBHOOKS — Inbound webhooks from SendGrid — no auth cookie, HMAC verified

POST `/webhooks/sendgrid`
Receives delivery status events from SendGrid

Security: Verify SendGrid's ECDSA signature in the X-Twilio-Email-Event-Webhook-Signature header before processing. Reject any request where the signature doesn't match.

SendGrid event payload (array of events)

Field	Type	Req?	Description
`event`	*string*	✓	Event type: delivered | bounce | open | click | spamreport
`email`	*string*	✓	Recipient email address
`timestamp`	*long*	✓	Unix timestamp of the event
`sg_message_id`	*string*	✓	SendGrid message ID — matches SendGridMessageId in SendMinutesResponseDto
`reason`	*string?*	—	Bounce reason (only present on bounce events)

What the handler does: Looks up the meeting by sg_message_id, updates the email delivery status in the DB, logs bounce reasons as warnings.

HTTP responses

Status	Meaning
200	Event acknowledged — always return 200 to SendGrid even if processing fails
400	Signature verification failed — event rejected

9. REQUEST DTOs — Full Reference — Every write DTO in one place

RegisterDto

Field	Type	Req?	Description
OrgName	*string*	✓	2–100 chars
Email	*string*	✓	Valid email, unique
Password	*string*	✓	Min 8 chars, 1 upper, 1 number, 1 special
ConfirmPassword	*string*	✓	Must equal Password
FullName	*string*	✓	2–100 chars

LoginDto

Field	Type	Req?	Description
Email	*string*	✓	Registered email
Password	*string*	✓	Account password

CreateMeetingDto

Field	Type	Req?	Description
Title	*string*	✓	3–255 chars
ScheduledAt	*DateTime?*	—	Defaults to DateTime.UtcNow
Attendees	*string[]?*	—	Pre-populate attendee list
Description	*string?*	—	Max 500 chars

UpdateMinutesDto

Field	Type	Req?	Description
DraftJson	*object*	✓	Full MinutesDraftDto object
EditSummary	*string?*	—	Note about changes — max 200 chars

ApproveMinutesDto

Field	Type	Req?	Description
FinalHtml	*string*	✓	Sanitised HTML from TipTap editor. Must not be empty.

SendMinutesDto

Field	Type	Req?	Description
Subject	*string?*	—	Defaults to 'Minutes: {MeetingTitle}'
AdditionalNote	*string?*	—	Prepended to email body. Max 1000 chars.
OverrideRecipients	*string[]?*	—	If set, replaces org recipient list for this send only

UpdateOrgDto

Field	Type	Req?	Description
`Name`	*string?*	—	2–100 chars. Omit to keep current name.

InviteMemberDto

Field	Type	Req?	Description
`Email`	*string*	✓	Valid email address
`FullName`	*string*	✓	2–100 chars
`Role`	*string?*	—	Member | OrgOwner. Default: Member

UpdateRecipientsDto

Field	Type	Req?	Description
`Recipients`	*RecipientInputDto[]*	✓	Full replacement list. Empty array clears all.

RecipientInputDto

Field	Type	Req?	Description
`Email`	*string*	✓	Valid email address
`Name`	*string*	✓	Display name

ChangePlanDto

Field	Type	Req?	Description
`Plan`	*string*	✓	Free | Pro | Enterprise
`Reason`	*string?*	—	Internal note — logged to audit trail

10. RESPONSE DTOs — Full Reference — Every read DTO in one place

AuthResponseDto

Field	Type	Description
`UserId`	*Guid*	ID of the authenticated/registered user
`OrgId`	*Guid*	ID of the user's organization
`Email`	*string*	User email
`FullName`	*string*	User display name
`Role`	*string*	Admin | OrgOwner | Member
`Plan`	*string*	Free | Pro | Enterprise

UserProfileDto

Field	Type	Description
`UserId`	*Guid*	User ID
`OrgId`	*Guid*	Organization ID
`Email`	*string*	Email address
`FullName`	*string*	Display name
`Role`	*string*	Admin | OrgOwner | Member
`Plan`	*string*	Free | Pro | Enterprise
`MeetingsUsedThisMonth`	*int*	Used count toward monthly cap
`MeetingsLimit`	*int*	5 for Free, -1 for Pro/Enterprise

MeetingDto

Field	Type	Description
`Id`	*Guid*	Meeting ID
`OrgId`	*Guid*	Organization ID
`Title`	*string*	Meeting title
`Status`	*string*	Pipeline status — see Enum Values section
`ScheduledAt`	*DateTime?*	Meeting date/time (UTC)
`CreatedAt`	*DateTime*	Record created at (UTC)
`CreatedByName`	*string*	Display name of the creator

MeetingDetailDto

Field	Type	Description
`Id`	*Guid*	Meeting ID
`OrgId`	*Guid*	Organization ID
`Title`	*string*	Meeting title
`Description`	*string?*	Meeting description
`Status`	*string*	Current pipeline status
`ScheduledAt`	*DateTime?*	Meeting date/time
`CreatedAt`	*DateTime*	Created at
`EndedAt`	*DateTime?*	When recording stopped
`AudioDurationSecs`	*int?*	Audio duration in seconds
`CreatedByName`	*string*	Creator name
`ApprovedByName`	*string?*	Name of approver (null until approved)
`ApprovedAt`	*DateTime?*	Approval timestamp
`HasTranscript`	*bool*	True once transcription complete
`HasMinutes`	*bool*	True once minutes generated

MinutesDto

Field	Type	Description
`Id`	*Guid*	Minutes record ID
`MeetingId`	*Guid*	Meeting ID
`Status`	*string*	pending_validation | approved | sent
`Version`	*int*	Edit version counter, starts at 1
`DraftJson`	*object*	MinutesDraftDto — AI structured output
`ApprovedHtml`	*string?*	Final HTML — null until approved
`ApprovedBy`	*Guid?*	Approver user ID
`ApprovedByName`	*string?*	Approver display name
`ApprovedAt`	*DateTime?*	Approval timestamp
`CreatedAt`	*DateTime*	When AI generated the minutes
`UpdatedAt`	*DateTime*	Last edited at

MinutesDraftDto

Field	Type	Description
MeetingTitle	*string*	Meeting title from transcript
MeetingDate	*string*	Date extracted from transcript
Attendees	*string[]*	Identified attendee names
AgendaItems	*string[]*	Agenda topics discussed
DiscussionPoints	*DiscussionItem[]*	Topic + summary pairs
Decisions	*string[]*	Explicit decisions made
ActionItems	*ActionItemDto[]*	Tasks with owner and optional due date
NextMeeting	*string?*	Next meeting details if mentioned

TranscriptDto

Field	Type	Description
Id	*Guid*	Transcript record ID
MeetingId	*Guid*	Meeting ID
RawText	*string*	Full combined transcript text
CreatedAt	*DateTime*	When transcription completed
Segments	*TranscriptSegmentDto[]*	Speaker-labelled segments

OrgDto

Field	Type	Description
Id	*Guid*	Organization ID
Name	*string*	Org name
Plan	*string*	Free | Pro | Enterprise
PlanRenewsAt	*DateTime*	Monthly reset date
MeetingsUsedThisMonth	*int*	Used count this month
MeetingsLimit	*int*	5 for Free, -1 for Pro/Enterprise
MemberCount	*int*	Active member count
CreatedAt	*DateTime*	Org creation date

SendMinutesResponseDto

Field	Type	Description
`SentAt`	*DateTime*	When email was dispatched
`RecipientCount`	*int*	Number of recipients
`SendGridMessageId`	*string*	For delivery tracking via webhook

PagedResult

Field	Type	Description
`items`	*T[]*	Array of items for this page
`totalCount`	*int*	Total records matching the query
`page`	*int*	Current page (1-based)
`pageSize`	*int*	Items per page
`totalPages`	*int*	Total number of pages

11. ENUM VALUES — All shared enumerations used across the API

MeetingStatus (meeting.Status field)

Value	Description
`created`	Meeting record created. No audio yet.
`uploading`	Audio upload in progress.
`transcribing`	Whisper is processing the audio file.
`generating_minutes`	LLM is structuring the transcript into minutes.
`pending_validation`	Minutes ready — waiting for user review in the editor.
`approved`	Minutes approved by a user. Ready to send.
`sent`	Minutes emailed to distribution list.
`failed`	Background job encountered an unrecoverable error.

PlanTier (org.Plan field)

Value	Description
`Free`	0 — 5 meetings/month. No bot, no email, no search. Default for new orgs.
`Pro`	1 — Unlimited meetings. Email distribution, search, export.
`Enterprise`	2 — Everything in Pro + custom branding, SSO, unlimited retention.

UserRole (user.Role field)

Value	Description
`Admin`	Platform administrator. Access to all /api/admin/* endpoints.
`OrgOwner`	Organization owner. Can manage members, recipients, and settings.
`Member`	Standard member. Can create meetings, validate and approve minutes.

MinutesStatus (minutes.Status field)

Value	Description
`pending_validation`	AI draft generated — not yet reviewed by a human.
`approved`	Human approved — locked for editing.
`sent`	Email dispatched to distribution list.

12. ERROR SHAPES — Standard error response body for all 4xx and 5xx

Standard error response body — every error returns this shape

```
{
  "error":       "error_code",    // machine-readable code (see table below)
  "message":     "Human-readable explanation of what went wrong",
  "requiredPlan": "Pro"           // ONLY present on 403 plan_required errors
}
```

All error codes

HTTP	error code	When it occurs
400	validation_error	Request body failed FluentValidation — check field errors
400	invalid_request	Request body is malformed or missing required fields
400	file_too_large	Audio file exceeds 100MB limit
400	unsupported_file_type	Audio file format not accepted (.mp3/.wav/.m4a/.webm/.ogg only)
400	empty_minutes	ApproveMinutesDto.FinalHtml is empty
400	minutes_already_sent	Attempted to send minutes that were already sent
401	unauthorized	No JWT cookie present or JWT is expired/invalid
402	plan_limit	Free plan: monthly meeting cap (5) has been reached
402	feature_requires_plan	Attempted to use a Pro/Enterprise feature on Free plan
403	plan_required	Endpoint requires a higher plan — response includes requiredPlan field
403	forbidden	Authenticated but insufficient role for this action
404	not_found	The requested resource does not exist or belongs to another org
409	email_already_exists	Registration: email is already in use
409	already_invited	Invite: email already has a pending invitation or is already a member
409	audio_already_uploaded	Audio upload: this meeting already has an audio file
409	already_approved	Attempted to approve minutes that are already approved
500	server_error	Unhandled exception — details logged server-side, not exposed to client
500	ai_service_error	Whisper or LLM API call failed after retries — meeting set to 'failed'
500	email_service_error	SendGrid API call failed — minutes remain 'approved', retry is safe